

Web Design — Study Notes

FINKI | Introduction to Web Design
Lessons 01–06 + Flexbox

Lesson 01 — Introduction to Web Design

How the Web Works

The Internet is a network of interconnected networks. The World Wide Web (WWW) is the part of the Internet concerned with displaying HTML documents. Standards are managed by the W3C (World Wide Web Consortium).

Web page delivery flow:

- Client (browser) sends a domain name request
- DNS server resolves domain to an IP address
- Browser contacts the web server at that IP
- Web server returns the HTML/CSS/JS files

Static vs Dynamic Pages

Type	Description	Technologies
Static	Same content for all users, no server processing	HTML, CSS
Dynamic	Content generated per user on the fly	PHP, Java, Python, C#/.NET + Database

HTML & CSS — The Basics

- **HTML** (HyperText Markup Language) — defines content and structure
- **CSS** (Cascading Style Sheets) — defines appearance and layout
- They are independent. CSS must NOT be written inside HTML and vice versa.
- Rule: HTML = content, CSS = appearance. Always.

HTML Terms

Term	Definition	Example
Element	A designator defining structure/content	<a>, <div>, <p>
Tag	Opening/closing brackets around element name	...
Attribute	Name/value pair on opening tag, adds info	href="url", class="name"
Self-closing	Element with no content, no closing tag	, , <meta />

CSS Terms

Term	Definition	Example
Selector	Targets which HTML element to style	p, .class, #id
Property	The style characteristic to change	color, font-size, margin
Value	The setting for the property	red, 16px, auto
Declaration	A property:value pair	color: red;
Rule set	Selector + block of declarations	p { color: red; }

CSS Selectors — Types

Selector	Targets	Specificity
p	All <p> elements (type/element)	(0,0,1)
.class	Elements with that class	(0,1,0)
#id	Element with that ID	(1,0,0)
*	Every element (universal)	(0,0,0)

Referencing CSS

- **External** (preferred): <link rel="stylesheet" href="style.css"> in <head>
- **Internal/Embedded**: <style> block inside <head>
- **Inline**: style attribute on individual element — avoid, highest specificity

Note: Always use external CSS for maintainability. CSS resets (e.g. Eric Meyer's reset) normalize browser default styles before you begin.

Lesson 02 — Introduction to HTML

Semantics

Semantics = giving content meaning and structure by using the *correct* element. Semantic HTML helps browsers, screen readers, and search engines understand your content.

Block vs Inline Elements

Type	Behaviour	Examples
Block-level	Starts on new line, takes full available width, can contain inline elements	<code><div></code> , <code><p></code> , <code><h1></code> , <code><h2></code> , <code><section></code>
Inline-level	Stays in flow, only as wide as content, cannot wrap block elements	<code></code> , <code><a></code> , <code></code> , <code></code> , <code></code>

div vs span

- **<div>** — block-level container, no semantic meaning, used for large layout groupings
- **** — inline container, no semantic meaning, used for styling small bits of text

HTML5 Semantic Structure Elements

Element	Purpose
<code><header></code>	Top of page or section — logo, nav, title. NOT the same as <code><head></code> .
<code><nav></code>	Navigation links — menus, breadcrumbs
<code><main></code>	Primary page content — only ONE per page
<code><section></code>	Thematic grouping of related content with a heading
<code><article></code>	Self-contained, independently distributable content (blog post, news)
<code><aside></code>	Tangentially related content — sidebars, pull quotes
<code><footer></code>	Closing content — copyright, links. New in HTML5.

Note: Before HTML5, developers used `<div id="header">`, `<div id="footer">` etc. HTML5 gives these proper semantic tags.

Deciding: article vs section vs div

- **<article>** — would the content make sense on its own, removed from the page? Use article.
- **<section>** — is this content part of a thematic group? Use section.
- **<div>** — no semantic reason at all, just layout grouping? Use div.

Text Elements

Element	Purpose
<code><h1></code> – <code><h6></code>	Headings — h1 is most important, h6 least. One h1 per page.
<code><p></code>	Paragraph of text
<code></code>	Bold with semantic importance

Element	Purpose
	Italic with semantic emphasis
	Bold — visual only, no semantic meaning
<i>	Italic — visual only, no semantic meaning
<small>	Side comments, fine print, copyright notices

	Line break (self-closing)
<hr>	Horizontal divider (self-closing)

Special Character Encoding

Special characters must be encoded in HTML. Format: **&name;** or **&#number;**

Character	Encoding
Copyright ©	&copy;
Em dash —	&mdash;
En dash –	&ndash;
Ampersand &	&amp;
Less than <	&lt;
Greater than >	&gt;
Non-breaking space	&nbsp;
é (e accent)	&eacute;

Hyperlinks

Type	Syntax	Notes
External link		Absolute path — full URL
Internal link		Relative path — relative to current file
New tab		Opens in new browser tab
Email		Opens email client
Phone		Calls on mobile
Anchor		Jumps to element with that ID on same page

Lesson 03 — Introduction to CSS

The Cascade

CSS stands for *Cascading* Style Sheets. Styles flow top to bottom — later rules override earlier ones for the same property on the same selector.

```
p { background: orange; font-size: 24px; }  
p { background: green; } /* green wins – comes later */
```

However, cascade order is overridden by **specificity**. A more specific selector wins regardless of order.

Specificity — The (A, B, C) System

Column	Counts	Points
A	ID selectors (#id)	(1,0,0)
B	Class (.class), attribute ([attr]), pseudo-class (:hover)	(0,1,0)
C	Element/type (p, div), pseudo-element (::before)	(0,0,1)
—	Universal selector (*), combinators	(0,0,0)
Inline style	style="" attribute on element	(1,0,0,0) — beats all
!important	Overrides all specificity	Nuclear option

Key rule: Compare left to right. One ID beats any number of classes. One class beats any number of elements. It is NOT simple addition across columns.

Specificity Examples

Selector	A	B	C	Score
p	0	0	1	(0,0,1)
.card	0	1	0	(0,1,0)
div.card	0	1	1	(0,1,1)
#header	1	0	0	(1,0,0)
#header p	1	0	1	(1,0,1)
div#container > ul	1	0	2	(1,0,2)
.hotdog p.mustard	0	2	1	(0,2,1)
#nav .item:hover	1	2	0	(1,2,0)

Combining Selectors (Combinators)

Combinator	Symbol	Meaning	Example
Descendant	space	Any descendant at any depth	div p — all <p> inside <div>
Child	>	Direct children only	div > p — only direct <p> children
Adjacent sibling	+	Immediately following sibling	h2 + p — <p> directly after <h2>

Combinator	Symbol	Meaning	Example
General sibling	~	All following siblings same parent	h2 ~ p — all <p> after <h2> in same parent

Colors in CSS

Format	Syntax	Example
Keyword	Named color	color: red;
Hexadecimal	#RRGGBB or #RGB (shorthand)	color: #ff6600; or color: #f60;
RGB	rgb(R, G, B) — 0 to 255 each	color: rgb(255, 102, 0);
RGBA	rgba(R, G, B, A) — A = opacity 0–1	color: rgba(255, 102, 0, 0.5);
HSL	hsl(hue, sat%, light%)	color: hsl(24, 100%, 50%);
HSLa	hsla(hue, sat%, light%, A)	color: hsla(24, 100%, 50%, 0.5);

Length Units in CSS

Unit	Type	Description
px	Absolute	Pixels — fixed size, most common absolute unit
%	Relative	Percentage of parent element's value
em	Relative	Relative to current element's font-size (1em = current font size)
rem	Relative	Relative to root (<html>) font-size — more predictable than em
vw	Relative	1% of viewport width
vh	Relative	1% of viewport height

Multiple Classes

Elements can have multiple class values separated by spaces. This allows modular, reusable styles:

```
<a class="btn btn-danger">Delete</a>
<a class="btn btn-success">Save</a>

.btn { font-size: 16px; }
.btn-danger { background: red; }
.btn-success { background: green; }
```

Lesson 04 — Complex CSS Selectors

Attribute Selectors

Selector	Meaning	Example	Matches
[attr]	Has the attribute	[open]	
[attr="val"]	Exactly equals val	[type="text"]	type="text" only
[attr~="val"]	Space-separated list contains val	[class~="small"]	class="big small"
[attr ="val"]	Equals val or starts with val-	[lang ="en"]	lang="en" or lang="en-US"
[attr^="val"]	Starts with val	[href^="https"]	href="https://..."
[attr\$="val"]	Ends with val	[href\$=".pdf"]	href="doc.pdf"
[attr*="val"]	Contains val anywhere	[href*="google"]	href="...google..."

Pseudo-Classes — Structural

Pseudo-class	Selects
:first-child	Element that is the first child of its parent
:last-child	Element that is the last child of its parent
:nth-child(n)	nth child counting from top. n can be number, odd, even, or formula like 2n+1
:nth-last-child(n)	nth child counting from BOTTOM
:nth-of-type(n)	nth element of that type among siblings
:nth-last-of-type(n)	nth of that type counting from bottom
:first-of-type	First element of that type among siblings
:last-of-type	Last element of that type among siblings
:only-child	Element that is the only child of its parent
:only-of-type	Only element of that type among siblings
:empty	Element with no children or text
:root	The root element (<html>)
:not(selector)	Every element that does NOT match the selector

nth-child Formula Guide

Formula pattern: **An+B** — A is the step, B is the offset, n starts at 0.

Formula	Selects
2n	Every even child (2, 4, 6, 8...)
2n+1	Every odd child (1, 3, 5, 7...)
3n	Every 3rd child (3, 6, 9...)
n+3	All children from 3rd onward

Formula	Selects
-n+3	First 3 children only (3, 2, 1)
-2n+3	From bottom: positions 1 and 3 (used with nth-last-child)
odd / even	Keyword shortcuts for 2n+1 / 2n

Pseudo-Classes — Link & UI State

Pseudo-class	Selects
:link	Unvisited links
:visited	Visited links
:hover	Element being hovered by mouse
:active	Element being clicked
:focus	Element that has keyboard/form focus
:checked	Checked checkbox or radio button
:enabled	Enabled form elements
:disabled	Disabled form elements

Pseudo-Elements

Pseudo-elements use :: (double colon). They create virtual elements not in the HTML.

Pseudo-element	Creates
::before	Virtual element inserted BEFORE element's content (requires content: property)
::after	Virtual element inserted AFTER element's content (requires content: property)
::first-line	Targets the first line of a block element
::first-letter	Targets the first letter of a block element

Cascade, Specificity & Inheritance Summary

Priority (lowest to highest)	
1. Browser default styles	Built-in browser stylesheet
2. External/linked stylesheet	<link rel="stylesheet">
3. Embedded stylesheet	<style> block in <head>
4. Inline styles	style="" attribute
5. !important	Overrides everything

Inheritance: Some CSS properties are inherited by children from their parent (e.g. color, font-size). Others are not (e.g. border, padding, margin). The **inherit** keyword forces a property to inherit its parent's value.

Lesson 06 — Positioning Content

Floats

The **float** property removes an element from normal document flow and positions it to the left or right of its parent. Other elements wrap around the floated element.

```
img { float: left; }
section { float: left; width: 63%; }
aside { float: right; width: 30%; }
```

Values: **left**, **right**, **none** (default)

Clearing Floats

After floating elements, subsequent elements may wrap unexpectedly. Use **clear** to stop wrapping:

```
footer { clear: both; }
```

Value	Effect
left	Clears left floats
right	Clears right floats
both	Clears both — use this almost always
none	Default — does not clear

Containing Floats (Clearfix)

An alternative to clearing is containing — wrap floated elements in a parent with the clearfix technique:

```
.group:before,
.group:after { content: ""; display: table; }
.group:after { clear: both; }
.group { clear: both; *zoom: 1; }
```

Then wrap your floated elements: `<div class="group"><section>...<aside>...</div>`

Inline-Block

display: inline-block allows elements to sit side by side (like inline) while still respecting width and height (like block). Useful alternative to floats for layout.

```
.column { display: inline-block; width: 30%; }
```

Note: Inline-block elements have whitespace sensitivity — gaps appear between elements because of whitespace in HTML. Fix by removing whitespace in HTML or setting font-size: 0 on parent.

CSS position Property

Value	Behaviour	Offset from
static	Default — normal document flow, ignores top/left/etc.	N/A
relative	Stays in flow but can be offset from its own normal position	Its own original position
absolute	Removed from flow, positioned relative to nearest positioned ancestor	Nearest positioned ancestor
fixed	Removed from flow, fixed to viewport — stays when scrolling	Viewport

Value	Behaviour	Offset from
sticky	Relative until scroll threshold, then fixed	Scroll container

Relative Positioning

Element stays in normal flow. Offset properties (top, right, bottom, left) shift it visually from its original position but the original space is preserved.

```
.box {  
  position: relative;  
  top: 20px;   /* moves DOWN 20px */  
  left: 40px;  /* moves RIGHT 40px */  
}
```

Absolute Positioning

Element is fully removed from normal flow. It is positioned relative to its nearest ancestor that has **position: relative** (or absolute/fixed). If none exists, it is positioned relative to the <body>.

```
.parent { position: relative; }  
.child  { position: absolute; top: 0; right: 0; }
```

Flexbox — Flexible Box Layout

Flexbox gives a container the ability to alter its items' width, height, and order to best fill available space. It is direction-agnostic — works both horizontally and vertically.

Two roles: the **flex container** (parent) and **flex items** (children). Properties are split between them.

Activating Flexbox

```
.container {  
  display: flex;           /* block-level flex container */  
  /* or */  
  display: inline-flex;    /* inline-level flex container */  
}
```

Container Properties (Parent)

flex-direction

Sets the main axis — the direction items are placed.

Value	Direction
row (default)	Left to right (horizontal)
row-reverse	Right to left
column	Top to bottom (vertical)
column-reverse	Bottom to top

flex-wrap

Controls whether items wrap onto multiple lines.

Value	Behaviour
nowrap (default)	All items on one line — may overflow
wrap	Items wrap to next line, top to bottom
wrap-reverse	Items wrap to next line, bottom to top

flex-flow

Shorthand for flex-direction + flex-wrap. Default: **row nowrap**

```
.container { flex-flow: row wrap; }
```

justify-content

Aligns items along the **main axis** (horizontal if row, vertical if column).

Value	Behaviour
flex-start (default)	Items packed at start of container
flex-end	Items packed at end of container
center	Items centered along main axis
space-between	Items evenly distributed; first at start, last at end

Value	Behaviour
space-around	Equal space around each item (half at edges)
space-evenly	Exactly equal spacing between all items including edges

align-items

Aligns items along the **cross axis** (perpendicular to main axis) — one line.

Value	Behaviour
stretch (default)	Items stretch to fill container height/width
flex-start	Items aligned at cross-start
flex-end	Items aligned at cross-end
center	Items centered on cross axis
baseline	Items aligned by their text baseline

align-content

Aligns **multiple lines** of items on the cross axis. Has no effect when there is only one line.

Value	Behaviour
stretch (default)	Lines stretch to fill remaining space
flex-start	Lines packed at start
flex-end	Lines packed at end
center	Lines centered
space-between	Lines evenly distributed
space-around	Equal space around each line

Item Properties (Children)

Property	Default	Description
order	0	Controls display order. Lower numbers appear first. Can be negative.
flex-grow	0	How much the item grows relative to others when there is extra space. 0 = no grow.
flex-shrink	1	How much the item shrinks relative to others when space is limited. 0 = no shrink.
flex-basis	auto	Initial size of item before space is distributed. Can be px, %, or auto.
flex	0 1 auto	Shorthand for flex-grow, flex-shrink, flex-basis. Recommended over individual props.
align-self	auto	Overrides align-items for this specific item. Same values as align-items.

flex-grow Example

If item A has flex-grow: 1 and item B has flex-grow: 2, B gets twice the extra space as A.

```
.item-a { flex-grow: 1; }
.item-b { flex-grow: 2; } /* gets 2x the leftover space */
```

Perfect Centering with Flexbox

The most common use case — centering an element both horizontally and vertically:

```
.container {  
  display: flex;  
  justify-content: center; /* horizontal */  
  align-items: center;    /* vertical */  
  height: 100vh;  
}
```

Full Flexbox Example

```
.flex-container {  
  display: flex;  
  flex-flow: row wrap;  
  justify-content: space-around;  
  padding: 0;  
  margin: 0;  
  list-style: none;  
}  
  
.flex-item {  
  width: 200px;  
  height: 150px;  
  margin-top: 10px;  
  background: tomato;  
  text-align: center;  
}
```

```
<ul class="flex-container">  
  <li class="flex-item">1</li>  
  <li class="flex-item">2</li>  
  <li class="flex-item">3</li>  
</ul>
```

Quick Reference — Common CSS Properties

Property	Values / Notes
color	Text color. Any color format.
background-color	Element background color.
background	Shorthand: color, image, repeat, position, size
font-size	px, em, rem, %. Default browser = 16px.
font-family	Font name. Always provide fallbacks: Arial, sans-serif
font-weight	normal (400), bold (700), or numeric 100–900
font-style	normal, italic, oblique
text-align	left, right, center, justify
text-decoration	none, underline, overline, line-through
text-transform	uppercase, lowercase, capitalize, none
text-shadow	offsetX offsetY blur color (e.g. 2px 2px 4px gray)
line-height	Unitless (1.5), px, or em. Controls spacing between lines.
letter-spacing	Space between characters. px or em.
width / height	px, %, auto, vw/vh
max-width / min-width	Constraints on width
margin	Space OUTSIDE element. margin: top right bottom left (clockwise)
padding	Space INSIDE element. Same shorthand as margin.
border	Shorthand: width style color (e.g. 1px solid red)
border-radius	Rounds corners. px or %. 50% = circle.
display	block, inline, inline-block, none, flex, grid
position	static, relative, absolute, fixed, sticky
top / right / bottom / left	Offset for positioned elements (not static)
z-index	Stack order. Higher = on top. Only works on positioned elements.
float	left, right, none
clear	left, right, both, none
overflow	visible (default), hidden, scroll, auto
opacity	0 (invisible) to 1 (fully visible)
cursor	pointer, default, text, crosshair, grab, not-allowed...
list-style-type	disc, circle, square, decimal, none
list-style-image	url("img.gif") — custom bullet image
box-shadow	offsetX offsetY blur spread color (e.g. 2px 2px 6px rgba(0,0,0,0.3))
transition	property duration timing-function delay